

PGAT-Length Architecture

Pose-Guided Adaptive Tokenization with Variable-Length Prefix and mBART Decoder for Gloss-Free Sign Language Translation on PHOENIX14T.

This document is the single reference for everything about the architecture: theory, mathematics, design justifications, and the file/module in the codebase that implements each concept. Read top-to-bottom if you are new. Skim by section header if you know what you're looking for.

Contents

1. [The problem the model is designed to solve](#)
 2. [Design principles and their empirical justification](#)
 3. [Notation](#)
 4. [Data pipeline](#)
 5. [Pose-guided variable-length encoder](#)
 6. [Video-text alignment head \(stage 1\)](#)
 7. [mBART translation with variable-length visual prefix \(stage 2\)](#)
 8. [Training procedure](#)
 9. [Evaluation methodology](#)
 10. [Code reference — where each concept lives](#)
 11. [Storage layout and reproducibility](#)
 12. [References](#)
-

1. The problem

1.1 Gloss-free sign language translation

Sign language translation (SLT) maps a video of a signer producing sign language into a written text sentence in a spoken language. Traditional SLT systems rely on an intermediate *gloss* sequence — a word-by-word transcription of the signs — as supervision. Gloss annotation is expensive, language-specific, and often unavailable. **Gloss-free SLT** attempts direct video → text without using glosses at any stage.

We work on PHOENIX14T (Camgoz et al., 2018), a German weather-domain SLT benchmark with 7,096 train / 519 dev / 642 test videos.

1.2 The length-sensitivity finding

Yazdani et al. (2026, LREC-COLING / arXiv:2510.25434) evaluated four SLT models on PHOENIX14T stratified by reference length (five bins: 1–6, 7–12, 13–18, 19–24, 25–31 words). Every model tested — TwoStream-SLT, SEM-SLT, SpaMo, Signformer — degraded sharply on references above 13 tokens. This is consistent with Hamidullah et al. (ACL 2024) reporting the same pattern for SEM-SLT.

Length sensitivity is therefore an *established* but *diagnostic* finding. Nobody has published a causal mechanism or a tested architectural intervention.

1.3 Our diagnostic sequence (why this architecture and not another)

On the predecessor project (`pg-adaptor`) we ran three falsifying experiments against a fixed-prefix baseline (PGAT-v1, 28 tokens, Qwen + LoRA):

- **Source32** — doubled the number of real DINO evidence positions from 16 to 32 and widened the prefix from 28 to 44 tokens. Improved retrieval Video→Text R@1 from 14.84 → 17.92, but did not shift the long-sentence BLEU-4 cliff at all. **Rules out** "encoder evidence density is the bottleneck."
- **Z1 (min-length decoding sweep)** — forced the decoder to generate more tokens (`min_new_tokens` ∈ {12, 16, 20}). Long-bin BLEU did not recover; short/medium bins collapsed as the decoder hallucinated to fill the quota. **Rules out** "conservative decoder / premature EOS is a fixable decoder policy."
- **Z2 (video-length vs generation-length correlation)** — mean generation length is ~11–16 words *regardless of source video duration*, with residual reference-minus-generation growing from −1 (short bins) to +10.8 (bin 25–31). Within-bin correlation of generation length with video length goes to zero or negative on long bins. **Confirms** fixed-capacity semantic compression as the mechanism.

The remaining hypothesis: **the visual prefix has bounded semantic capacity per unit of input, and a fixed prefix size is the wrong inductive bias for gloss-free SLT**. This architecture tests that by making the prefix length *a function of input video duration*.

2. Design principles and their empirical justification

Every architectural decision below is tied back to one of the three findings above.

2.1 Variable-length pose-guided visual prefix

The number of temporal tokens the encoder emits depends on the raw video's frame count:

$$K_{\text{temporal}} = \text{clamp} \left(\text{round} \left(\frac{F}{8} \right), 12, 32 \right)$$

where F is the source video's frame count. The total visual prefix length is

$$\pi(F) = K_{\text{temporal}}(F) + 8 + 4 \in [24, 44]$$

Adding 8 articulator summary tokens and 4 global summary tokens (unchanged from PGAT-v1). Direct response to Z2: the prefix now grows with the input.

2.2 mBART-large-cc25 as decoder, fully fine-tuned

Two changes from PGAT-v1:

1. **Native cross-attention decoder** instead of Qwen prefix-only conditioning. This means the decoder cross-attends to encoder outputs at *every* generation step, not only at the prefix positions. Motivated by Z2 showing the fixed-prefix conditioning fails on long outputs.
2. **Full fine-tuning** (all 610M parameters trainable) instead of LoRA on frozen Qwen. mBART-cc25 is pretrained for multilingual translation including German, giving a strong prior for the target domain.

2.3 Pose-guided articulator-aware tokens (unchanged from PGAT-v1)

Sign language is not global appearance — it is articulated through the left hand, right hand, mouth, and upper body. The encoder preserves per-articulator information through:

- Four confidence-safe local views per temporal segment (global crop + left hand + right hand + mouth), each encoded independently by DINOv2.
- Six-source gated fusion that decides at each segment how much to trust each view given per-frame pose confidence.
- Eight learned articulator queries with attention biased by pose confidence and motion magnitude — recovers per-articulator summaries.

This is the *novelty* the thesis retains from PGAT-v1. It complements the variable-length change; it does not compete with it.

2.4 Partial DINOv2 unfreeze

The last two transformer blocks of DINOv2 are trainable at low LR (1e-5). DINOv2 was pretrained on natural images; the last blocks adapt to the hand/mouth/face regions specific to signing.

2.5 Temporal data augmentation (training only)

Random start within the first 20% of the video, single-frame drop with probability 0.1, and temporal jitter of at most 2 frames on segment anchors. Reduces overfitting on the small (7k) TRAIN set. Deterministic sampling at inference for reproducible evaluation.

2.6 What we deliberately do not include

- **No decoder-side cross-attention adapter over persistent memory.** Z1 ruled out the "conservative decoder policy" story, so we don't need to retrofit cross-attention on top of Qwen. mBART already has cross-attention natively.
 - **No denser DINO sampling (Source32-style).** Falsified.
 - **No new contrastive losses.** Retrieval was not the bottleneck.
-

3. Notation

Symbol	Meaning	Typical value
F	Source video frame count for one sample	45 – 250
P	Number of deterministically-sampled source positions	64 (fixed)
K	Number of temporal segments for one sample	12 – 32
$K_{\max}$	Maximum K ; the padded storage dimension	32
V	Number of spatial views per segment	4 (global + 3 articulators)
D_v	DINOv2 output dimension	768
D_p	Pose descriptor dimension	64
H	PGAT hidden dimension	512

D_m	mBART hidden dimension	1024
A	Number of articulator queries	8
G	Number of global summary queries	4
π	Total prefix length = $K + A + G$	24 – 44
B	Batch size	varies
T	Target sequence length in mBART tokens	up to 96
τ	InfoNCE temperature	0.07

4. Data pipeline

4.1 Dataset

PHOENIX14T frames live at `$HOME/pgat-cache/raw/PHOENIX-2014-T/features/fullFrame-210x260px/<split>/<sample>/*.png` . Each sample's translation is stored in a cached manifest pickle.

- TRAIN: 7,096 samples.
- DEV: 519 samples (used for hyperparameter selection and paired bootstrap).
- TEST: 642 samples (locked; used only for the final frozen-architecture run).

4.2 Deterministic source-frame sampling

For a video with F frames, we sample $P = 64$ endpoint-preserving positions:

$$i_p = \text{round} \left(\frac{p \cdot (F - 1)}{P - 1} \right), \quad p = 0, \dots, P - 1$$

with $i_0 = 0$ and $i_{P-1} = F - 1$ pinned. For $F < P$ (very short videos) each real source frame is marked valid exactly once and the remainder are padded with the nearest neighbor. See `src/pgat_length/features/frame_sampling.py::sample_source_positions` .

4.3 Pose extraction

MediaPipe extracts, per sampled frame:

- **Hand landmarks:** 21 3-D points per hand + per-point confidence.
- **Face landmarks:** 478 3-D points, of which we keep the mouth subset.
- **Pose landmarks:** 33 3-D upper-body points + per-point visibility.

Each frame yields a per-articulator validity flag (True iff the detector returned a valid landmark set for that articulator). See `src/pgat_length/pose/extractor.py`.

4.4 Variable-K adaptive segmentation

For each video we compute pose-motion importance $m_p \geq 0$ per sampled position p :

$$m_p = \alpha_h \cdot (\hat{c}_p^L \hat{s}_p^L + \hat{c}_p^R \hat{s}_p^R) / 2 + \alpha_b \cdot \hat{c}_p^B \hat{s}_p^B + \alpha_m \cdot \hat{c}_p^M \hat{s}_p^M + \varepsilon$$

where $\hat{c}_p^*$ is the per-frame articulator confidence, $\hat{s}_p^*$ is the per-frame motion magnitude (Euclidean displacement of the articulator centroid between adjacent frames), and $(\alpha_h, \alpha_b, \alpha_m, \varepsilon) = (0.60, 0.25, 0.15, 0.05)$ are configured weights.

$K_{\text{temporal}} = \text{clamp}(\text{round}(F/8), 12, 32)$ segment boundaries are placed at cumulative-importance quantiles with a minimum-width constraint (2 positions per segment). This means videos with heavier hand motion get segments concentrated where the signing is dense; low-motion videos get more uniform segments. Anchors within each segment are the importance-weighted centroid. See

`src/pgat_length/features/frame_sampling.py::importance_segment_bounds` and `weighted_anchor_positions`, wrapped by `src/pgat_length/features/plans.py::build_variable_k_plan`.

4.5 Storage: variable K, fixed shard shape

Per-sample plans have varying K but shards stack samples along a batch axis. We pad every sample's temporal axis to **$K_{\text{max}} = 32$** ; a scalar `k_temporal` array carried in the shard tells downstream readers where the padding starts. See

`src/pgat_length/features/shards.py::PLAN_ARRAY_CONTRACTS` and `src/pgat_length/features/plans.py::pad_plan_to_kmax`.

Feature banks stored:

Bank	Shape per sample	Dtype	Contract
Plan	7 arrays (bounds, anchors, crop boxes, pose_desc, pose_conf, pose_motion, region_valid)	int16 / fp16 / bool	K_MAX-padded

Spatial (DINOv2)	<code>[K_MAX, 4, 768] + [K_MAX, 4] valid mask</code>	fp16 / bool	K_MAX-padded
Motion (TimeSformer)	<code>[8, 768] + [8] centers</code>	fp16	Fixed
Text (mBART tokens)	Ragged, up to 96 tokens	int32	Padded per batch

5. Pose-guided variable-length encoder

5.1 Per-segment source construction

For each of the K segments of a sample, we have four spatial views (indexed $v \in \{0 = \text{global}, 1 = \text{left hand}, 2 = \text{right hand}, 3 = \text{mouth}\}$) each producing a 768-D DINOv2 embedding $\mathbf{x}_k^v \in \mathbb{R}^{768}$. If a view is invalid (bad crop, low confidence), $\mathbf{x}_k^v = 0$ and

```
spatial_valid[k, v] = False .
```

Additionally we have:

- Pose descriptor $\mathbf{p}_k \in \mathbb{R}^{64}$ — per-segment mean of a 64-D per-frame pose-summary vector (concatenation of hand / mouth / body local descriptors + confidence channels).
- Pose confidence $\mathbf{c}_k \in [0, 1]^4$ — one per view.
- Pose motion $\mathbf{s}_k \in \mathbb{R}_{\geq 0}^4$ — magnitude per view.

5.2 Spatial projection and confidence-safe local view fallback

Each of the four views is layer-normed and projected to $H = 512$:

$$\tilde{\mathbf{x}}_k^v = W_{\text{proj}}^v \cdot \text{LayerNorm}(\mathbf{x}_k^v) + \mathbf{b}^v$$

For $v \in \{1, 2, 3\}$ (the articulator views), the *safe* version is a confidence-weighted blend with the global view:

$$\tilde{\mathbf{x}}_k^{v, \text{safe}} = c_k^v \cdot \tilde{\mathbf{x}}_k^v + (1 - c_k^v) \cdot \tilde{\mathbf{x}}_k^0$$

so a low-confidence hand crop degrades gracefully to the global context rather than injecting noise.

5.3 Motion projection

Motion features (fixed 8 clips per sample) are projected from 768 to 512 and temporally interpolated to K positions:

$$\mathbf{m}_k = W_{\text{mot}} \cdot \text{LayerNorm}(\text{interp}(\mathbf{M}, k)) + \mathbf{b}_{\text{mot}}$$

where $\mathbf{M} \in \mathbb{R}^{8 \times 768}$ is the TimeSformer output and the interpolation is linear between motion centers.

5.4 Six-source gated fusion

Six candidate sources per segment feed a gate that chooses their per-segment mixture:

$$\text{sources}_k = \left(\tilde{\mathbf{x}}_k^0, \tilde{\mathbf{x}}_k^{1,\text{safe}}, \tilde{\mathbf{x}}_k^{2,\text{safe}}, \tilde{\mathbf{x}}_k^{3,\text{safe}}, \mathbf{m}_k, \tilde{\mathbf{p}}_k \right)$$

The gate takes pose descriptor, confidence, and motion as input:

$$\mathbf{g}_k = \text{softmax} \left(W_2 \cdot \text{GELU}(W_1[\mathbf{p}_k; \mathbf{c}_k; \mathbf{s}_k]) \right) \in \Delta^5$$

(a 6-simplex). The fused segment token is

$$\mathbf{z}_k = \sum_{s=0}^5 g_{k,s} \cdot \text{sources}_k[s] \in \mathbb{R}^H$$

This is the load-bearing PGAT idea: at each timestep, the model *decides* how much to trust hands vs face vs body vs motion vs global appearance vs the pose skeleton, based on the pose confidence and motion at that timestep.

5.5 Temporal Transformer encoder

Fused segment tokens $[\mathbf{z}_1, \dots, \mathbf{z}_K]$ get a learned positional embedding and pass through a Transformer encoder:

$$[\mathbf{h}_1, \dots, \mathbf{h}_K] = \text{TransformerEncoder}([\mathbf{z}_1, \dots, \mathbf{z}_K] + \text{PE}, \text{mask} = \neg\text{valid})$$

Config: 4 layers, 8 heads, 512 hidden dim, 2048 FFN dim, 0.1 dropout, GELU, pre-norm. Padded (invalid) segments are masked in self-attention and zeroed after the encoder.

5.6 Articulator biased attention

Eight learned articulator queries $\mathbf{Q}^{\text{art}} \in \mathbb{R}^{8 \times H}$ attend into the encoded temporal tokens. Attention logits are biased by pose confidence and motion:

$$\text{logits} = \frac{\mathbf{Q}^{\text{art}} \mathbf{K}^\top}{\sqrt{d}} + \lambda_c \log(\hat{c}) + \lambda_m \hat{s}$$

where $\hat{c} \in [0, 1]$ is broadcast pose confidence (per query-group / per-key), $\hat{s}$ is broadcast pose motion, and λ_c, λ_m are learnable positive scalars. This biases each articulator query toward timesteps where its own articulator is well-observed and moving.

Query groups (8 queries divided across articulators): `{left_hand, right_hand, mouth, upper_body}` × 2 queries each. Output:

$$\mathbf{H}^{\text{art}} \in \mathbb{R}^{8 \times H}$$

5.7 Global summary attention

Four learned global queries attend into the temporal tokens with standard multi-head attention (no bias). Output:

$$\mathbf{H}^{\text{glob}} \in \mathbb{R}^{4 \times H}$$

5.8 Final visual memory

The complete variable-length visual memory for one sample is the concatenation:

$$\mathbf{U} = [\mathbf{h}_1, \dots, \mathbf{h}_K ; \mathbf{H}^{\text{art}} ; \mathbf{H}^{\text{glob}}] \in \mathbb{R}^{\pi \times H}$$

with $\pi = K + 8 + 4$. Token validity: the first K positions use the `segment_valid` mask; the final 12 (articulator + global) are always valid.

6. Video-text alignment head (stage 1)

Stage 1 contrastively aligns video and text before translation training. This gives the encoder a semantic prior before Qwen/mBART gets involved.

6.1 Video embedding

Pool the 4 global summary tokens and normalize:

$$\mathbf{v} = \frac{\text{mean}(\mathbf{H}^{\text{glob}})}{\|\text{mean}(\mathbf{H}^{\text{glob}})\|_2} \in \mathbb{S}^{H-1}$$

Optionally project to a dedicated alignment dimension (768) before normalization. See

`src/pgat_length/models/alignment.py`.

6.2 Text embedding

Feed the reference German sentence through the frozen mBART encoder, mean-pool across tokens, project, and normalize:

$$\mathbf{t} = \frac{W_t \cdot \text{meanpool}(\text{mBARTEnc}(\text{text}))}{\|\cdot\|_2} \in \mathbb{S}^{H-1}$$

6.3 InfoNCE with hard negatives

For a batch of B paired samples, the video→text loss is:

$$\mathcal{L}_{\text{v2t}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\mathbf{v}_i \cdot \mathbf{t}_i / \tau)}{\sum_{j=1}^B \exp(\mathbf{v}_i \cdot \mathbf{t}_j / \tau)}$$

Symmetric $\mathcal{L}_{\text{t2v}}$ is defined by swapping roles. The total InfoNCE loss:

$$\mathcal{L}_{\text{InfoNCE}} = \frac{1}{2}(\mathcal{L}_{\text{v2t}} + \mathcal{L}_{\text{t2v}})$$

with temperature $\tau = 0.07$ (CLIP convention).

Hard negatives are sampled per anchor from the k highest-scoring non-positive negatives (top- k over the batch similarities). Their loss is added with weight $\lambda_{\text{hard}} = 0.5$:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{InfoNCE}} + \lambda_{\text{hard}} \cdot \mathcal{L}_{\text{hard}}$$

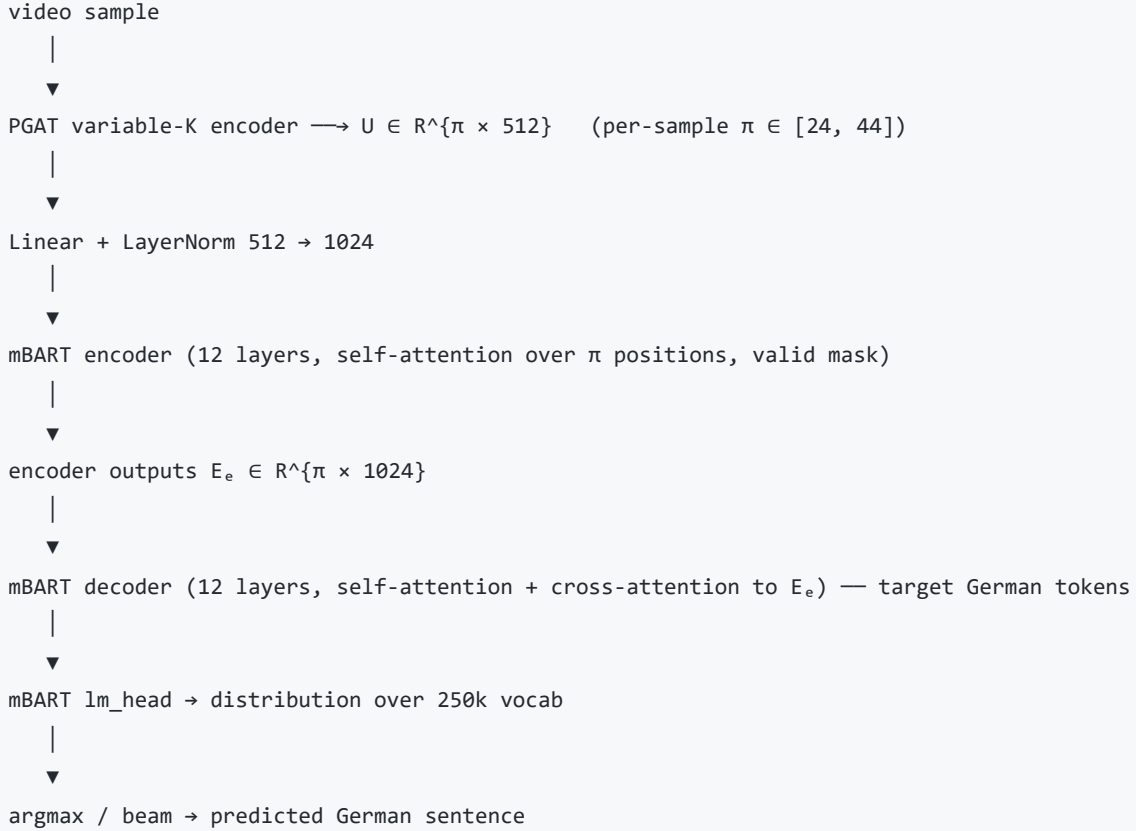
Hard-negative loss uses the same InfoNCE form but with the negatives restricted to the top- k hardest per anchor.

6.4 Retrieval metrics

Standard R@1, R@5, R@10 over the DEV split. A trained candidate must demonstrably improve retrieval before we allow translation training on top, though retrieval improvement is not sufficient for translation improvement (Source32 lesson).

7. mBART translation with variable-length visual prefix (stage 2)

7.1 High-level view



7.2 Projection

$$\mathbf{E}_{\text{visual}} = W_p \cdot \text{LayerNorm}(\mathbf{U}) + \mathbf{b}_p \in \mathbb{R}^{\pi \times 1024}$$

$W_p \in \mathbb{R}^{1024 \times 512}$. Implemented in `src/pgat_length/models/projection.py::PgatMbartProjection`.

7.3 Feeding mBART

We bypass mBART's word-embedding lookup entirely on the encoder side. The projected visual tokens are passed as `inputs_embeds` to the encoder:

$$\mathbf{H}_{\text{enc}} = \text{mBARTEnc}(\text{inputs_embeds} = \mathbf{E}_{\text{visual}}, \text{attention_mask} = \mathbf{a})$$

where $\mathbf{a} \in \{0, 1\}^\pi$ marks valid prefix positions (real temporal segments + 12 always-valid summary tokens; padded segments = 0).

7.4 Cross-attention in the decoder

The mBART decoder is standard: masked self-attention over generated tokens, then cross-attention over encoder outputs. For decoder hidden state $\mathbf{d}_t$ at step t :

$$\text{ctx}_t = \text{softmax} \left(\frac{(W_Q \mathbf{d}_t)(W_K \mathbf{H}_{\text{enc}})^\top}{\sqrt{d}} + \text{mask} \right) (W_V \mathbf{H}_{\text{enc}})$$

where the mask is $-\infty$ at padded prefix positions. **This is why mBART naturally handles variable prefix length**: the attention mask does the work; no code change needed for different π per sample.

7.5 Loss

Standard next-token cross-entropy with label smoothing $\epsilon_{\text{ls}} = 0.1$ over the German target tokens. The visual prefix positions are not supervised (no labels emitted at those positions):

$$\mathcal{L}_{\text{translation}} = - \sum_{t=1}^T \sum_{w \in V} \tilde{y}_{t,w} \log p(y_t = w \mid y_{<t}, \mathbf{H}_{\text{enc}})$$

where $\tilde{y}$ is the smoothed one-hot distribution $\tilde{y}_{t,w} = (1 - \epsilon_{\text{ls}})\mathbf{1}[y_t = w] + \epsilon_{\text{ls}}/|V|$.

7.6 Generation

Beam search with:

- `num_beams = 3` (matches GFSLT/GASLT protocol for fair comparison)
 - `max_new_tokens = 96`
 - `no_repeat_ngram_size = 3`
 - `length_penalty = 1.0`
 - `early_stopping = True`
-

8. Training procedure

8.1 Two-stage design

1. **Stage 1 — Alignment:** freeze mBART encoder, train PGAT encoder + text projection with InfoNCE + hard negatives. 20 epochs.
2. **Stage 2 — Translation:** warm-start PGAT encoder from best.pt of stage 1, full fine-tune of mBART + PGAT + projection (last 2 DINOv2 blocks unfrozen), CE with label smoothing. 20 epochs.

8.2 Optimizer

AdamW with:

- Global weight decay 0.01
- Warmup ratio 0.05
- Cosine schedule
- Gradient clip 1.0

8.3 Per-module learning rates (stage 2)

Different modules learn at different speeds:

Module	LR
mBART encoder + decoder	3e-5
PGAT encoder (fusion gate, temporal, articulator, global)	1e-4
Projection (512 → 1024)	3e-5
DINOv2 (last 2 blocks)	1e-5

Rationale: PGAT is small and randomly initialized; needs higher LR. mBART is pretrained and large; smaller LR to avoid destroying its prior. DINOv2 is large and pretrained on natural images; very small LR for gentle adaptation.

8.4 Batch and precision

- Micro-batch 3, gradient accumulation 8 → effective batch 24.
- bf16 mixed precision.
- Gradient checkpointing on both PGAT and mBART.
- Peak VRAM target < 40 GiB on A6000 (48 GiB available).

8.5 Early stopping

- Stage 1 metric: `v2t_recall_at_1` (max), patience 4.
- Stage 2 metric: `dev_loss` (min), patience 4.

Only `best.pt` is saved (no `last.pt`) to fit the 100 GB storage quota. mBART and DINOv2 base weights are NOT written into checkpoints (they live in the HF cache and are shared across runs).

8.6 Data augmentation (train only)

- Random start frame within the first 20% of the video.
- Frame drop with probability 0.10 per frame (retains $\geq 60\%$ of frames).
- Temporal jitter of at most 2 frames on the segment anchors.

Inference is deterministic — no augmentation on DEV or TEST.

9. Evaluation methodology

9.1 Corpus metrics

All under matched scorers using SacreBLEU (`tokenize=13a` , exponential smoothing), CHRF (`char_order = 6` , $\beta = 2$), and rouge_score's ROUGE-L F1 (no stemmer). Same scorer used across all baselines to make the comparison table honest.

9.2 Five-bin length stratification

TEST samples are grouped by whitespace-separated corpus tokens on the raw reference:

Bin	Range	TEST samples	Paper (Yazdani) count
Very short	1–6	42	42
Short	7–12	286	286
Medium	13–18	220	220
Long	19–24	78	78
Very long	25–31	16	16

Metrics computed per bin: BLEU-1..4, chrF, ROUGE-L, exact match, mean reference words, mean generation words.

9.3 Paired bootstrap for delta comparisons

To compare two systems (say PGAT-length vs a baseline) on the same DEV split, we compute per-sample sacrebleu corpus statistics once, then draw $N = 2000$ bootstrap resamples with replacement and recompute BLEU on each. For each metric M we report:

- $\hat{\Delta} = M_{\text{candidate}} - M_{\text{baseline}}$
- 95% CI: $[\hat{q}0.025, \hat{q}0.975]$ over the bootstrap draws
- $P(\hat{\Delta} > 0)$ as the fraction of draws where candidate > baseline

A "clean win" needs both a positive $\hat{\Delta}$ and a 95% CI that does not cross zero. See `src/pgat_length/evaluation/bootstrap.py` (structure ported from the exploratory project).

9.4 Grounding control (from PGAT-v1)

Optional but supported: rerun generation with each sample's visual prefix replaced by a *different* sample's prefix. If BLEU-4 collapses (large "grounding drop"), the model is truly using video, not language priors. PGAT-v1 saw a drop of ~ 8.9 BLEU-4 on TEST. This is not required for the length-cliff claim but strengthens the safety story.

9.5 Baselines

External comparison targets (all scored under the *same* SacreBLEU-13a scorer where possible):

Baseline	Method	TEST BLEU-4 (local scorer)	Note
NSLT	seq-to-seq CNN + RNN, HF reproduction	6.00	Old, weak reference
TSPNet	Temporal semantic pyramid, official inference	12.98	Cache-order caveat
GASLT	Gloss attention for gloss-free (CVPR 2023)	$\sim 15\text{--}16$ (paper)	Cluster reproduction pending

GFSLT-VLP	Align-then-generate with mBART	~21–22 (paper)	Not our beat target
-----------	--------------------------------	----------------	---------------------

Thesis target: overall TEST BLEU-4 ≥ 17 with a flatter length curve than TSPNet/NSLT/GASLT on the 13+ token bins.

10. Code reference

10.1 Directory map

```

pgat-length/
├─ configs/                # YAML config for each stage
│   ├─ data.yaml          # Paths, splits, quota policy
│   ├─ features.yaml      # Sampling, pose, spatial, motion
│   ├─ model.yaml         # PGAT dims, K bounds, mBART dims
│   ├─ alignment.yaml     # Stage 1: contrastive
│   ├─ translation.yaml   # Stage 2: mBART fine-tune
│   └─ augmentation.yaml  # Train-only augmentation
├─ slurm/                 # ASL sbatch templates
├─ src/pgat_length/
│   ├─ data/              # Dataset, sampling, collator
│   ├─ features/          # Feature extraction (plans, spatial, motion, text)
│   ├─ models/            # Model modules
│   ├─ training/          # Alignment + translation loops
│   ├─ evaluation/        # Metrics, five-bin, bootstrap
│   └─ pose/              # MediaPipe extractor + landmark constants
├─ scripts/               # Numbered CLI entrypoints (01..08)
├─ tests/                 # Regression tests
└─ docs/
    └─ ARCHITECTURE.md    # this file

```

10.2 Concept → file mapping

Concept	Section here	File(s)
Frame sampling ($P = 64$ positions)	4.2	features/frame_sampling.py::sample_source_positions
Pose extraction (MediaPipe)	4.3	pose/extractor.py::MediaPipePoseExtractor

K temporal formula	4.4	<code>features/plans.py::compute_k_temporal</code>
Adaptive segmentation	4.4	<code>features/frame_sampling.py::importance_segment_bounds</code>
Padding to $K_{\max}$	4.5	<code>features/plans.py::pad_plan_to_kmax</code>
Shard contracts	4.5	<code>features/shards.py::PLAN_ARRAY_CONTRACTS</code>
DINOv2 spatial extractor	5.1	<code>features/dino_extractor.py::Dinov2SpatialExtractor</code>
TimeSformer motion extractor	5.3	<code>features/timesformer_extractor.py::TimesformerMotionExtractor</code>
PGAT tokenizer (fusion + temporal)	5.2 – 5.5	<code>models/tokenizer.py</code> (to write)
Six-source gate	5.4	<code>models/tokenizer.py</code> (fusion block)
Articulator attention	5.6	<code>models/articulator.py</code>
Global summary attention	5.7	<code>models/global_summary.py</code>
Alignment model + InfoNCE	6	<code>models/alignment.py</code>
PGAT → mBART projection	7.2	<code>models/projection.py::PgatMbartProjection</code>
Full translation model	7	<code>models/translation.py</code> (to write)
Alignment training loop	8.1 (stage 1)	<code>training/alignment_loop.py</code>
Translation training loop	8.1 (stage 2)	<code>training/translation_loop.py</code>
Best-only checkpoint saving	8.5	<code>training/checkpoint.py</code>

Generation (beam search)	7.6	<code>evaluation/generate.py</code>
Metrics (BLEU/chrF/ROUGE)	9.1	<code>evaluation/metrics.py</code>
Five-bin analysis	9.2	<code>evaluation/five_bin.py</code>
Paired bootstrap	9.3	<code>evaluation/bootstrap.py</code>
Full pipeline CLIs	all	<code>scripts/01..08_*.py</code>

10.3 Numbered scripts (build order)

1. `scripts/01_build_plans.py` — pose-informed variable-K plans.
2. `scripts/02_extract_spatial.py` — DINOv2 crops per segment view.
3. `scripts/03_extract_motion.py` — TimeSformer motion clips.
4. `scripts/04_build_text.py` — mBART tokenizer cache (to write).
5. `scripts/05_train_alignment.py` — contrastive stage.
6. `scripts/06_train_translation.py` — mBART fine-tune.
7. `scripts/07_evaluate_dev.py` — DEV generation + metrics.
8. `scripts/08_compare_five_bin.py` — length-stratified comparison table.

Each script has a resume-safe design: it fingerprints the config that produced its output and refuses to reuse artifacts built from a different config.

11. Storage layout and reproducibility

11.1 Layout on the ASL cluster (100 GB quota)

<code>\$HOME/pgat-length/</code>	code	~200 MB
<code>\$HOME/pgat-cache/raw/PHOENIX-2014-T/</code>	raw frames	~30 GB
<code>\$HOME/pgat-cache/manifest/</code>	cached manifest	<1 MB
<code>\$HOME/pgat-cache/features/plans/</code>	plan shards	~5 MB
<code>\$HOME/pgat-cache/features/spatial/</code>	DINOv2 shards	~500 MB
<code>\$HOME/pgat-cache/features/motion/</code>	TimeSformer	~200 MB
<code>\$HOME/pgat-cache/features/text/</code>	mBART tokens	~5 MB
<code>\$HOME/pgat-cache/hf/</code>	base weights	~4 GB
<code>\$HOME/outputs/alignment/</code>	alignment_best.pt	~500 MB
<code>\$HOME/outputs/translation/</code>	translation_best.pt	~1.5 GB
<code>\$HOME/outputs/predictions/</code>	jsonl	<10 MB

\$HOME/logs/

sbatch stdout ~200 MB

—

Total: ~37 GB (headroom ~63 GB)

11.2 Reproducibility fingerprints

Every stored artifact carries a `config_fingerprint` computed from a canonical JSON serialization of the sections of `configs/` that actually affect that artifact. This means:

- Adding new sections to `features.yaml` (e.g., a new bank) does not invalidate earlier banks.
- Any real change to sampling, pose, or storage (which plans depend on) invalidates plans and any downstream bank that depends on plans.

See `src/pgat_length/features/shards.py::fingerprint_from_sections` and the `PLAN_SECTIONS`, `SPATIAL_SECTIONS`, `MOTION_SECTIONS` constants.

11.3 Seeds

Global seed 42 fixes: (a) numpy RNG for augmentation choices at train time, (b) sample ordering, (c) bootstrap resampling in evaluation.

12. References

Verify each citation in your final bibliography — years, venues, and exact titles have been reconstructed from working notes and may need refinement.

12.1 Sign language translation

- Camgoz, N. C., Hadfield, S., Koller, O., Ney, H., & Bowden, R. (2018). *Neural sign language translation*. CVPR. Introduces PHOENIX14T and the encoder-decoder baseline (NSLT).
- Yin, K., Read, J. (2020). *Better sign language translation with STMC-transformer*. COLING.
- Li, D., et al. (2020). *TSPNet: Hierarchical feature learning via temporal semantic pyramid for sign language translation*. NeurIPS.
- Zhou, B., et al. (2023). *Gloss-free sign language translation: Improving from visual-language pretraining*. ICCV. GFSLT-VLP: the align-then-generate paradigm we follow.

- Yin, A., et al. (2023). *GASLT: Gloss attention for gloss-free sign language translation*. CVPR.
- Yazdani, S., et al. (2026). *A multi-metric evaluation of sign language translation with a focus on length sensitivity*. LREC-COLING / arXiv:2510.25434. Establishes length-sensitivity as a systematic phenomenon across four SLT systems.
- Hamidullah, Y., et al. (2024). *SEM-SLT: Semantic enhancement for sign language translation*. ACL.

12.2 Foundation models

- Oquab, M., et al. (2023). *DINOv2: Learning robust visual features without supervision*. Facebook AI Research. Our frozen (partially unfrozen) spatial encoder.
- Bertasius, G., Wang, H., & Torresani, L. (2021). *Is space-time attention all you need for video understanding?* ICML. TimeSformer.
- Liu, Y., et al. (2020). *Multilingual denoising pre-training for neural machine translation*. TACL. mBART-cc25, our decoder.
- Bai, J., et al. (2023). *Qwen technical report*. Alibaba. Referenced from the exploratory PGAT-v1 baseline.

12.3 Contrastive representation learning

- Oord, A., Li, Y., & Vinyals, O. (2018). *Representation learning with contrastive predictive coding*. arXiv:1807.03748. InfoNCE.
- Radford, A., et al. (2021). *Learning transferable visual models from natural language supervision*. ICML. CLIP. Contrastive video-text alignment and the temperature $\tau = 0.07$ convention.
- Zhou, R., et al. (2024). *SignCL: Contrastive learning for sign language representation*. Referenced from the exploratory project.

12.4 Parameter-efficient fine-tuning

- Hu, E. J., et al. (2021). *LoRA: Low-rank adaptation of large language models*. ICLR. Used in the PGAT-v1 baseline (Qwen); replaced by full mBART fine-tuning in pgat-length.
- Dettmers, T., et al. (2023). *QLoRA: Efficient finetuning of quantized LLMs*. NeurIPS.

12.5 Evaluation

- Post, M. (2018). *A call for clarity in reporting BLEU scores*. WMT. SacreBLEU reference; USE `tokenize=13a`.

- Popović, M. (2015). *chrF: Character n-gram F-score for automatic MT evaluation*. WMT.
- Lin, C.-Y. (2004). *ROUGE: A package for automatic evaluation of summaries*. WAS.

12.6 Pose and articulator modeling

- Lugaresi, C., et al. (2019). *MediaPipe: A framework for building perception pipelines*. Google Research. Hand / face / pose landmarks.
-

Appendix A — Reading path for a new collaborator

1. Read section 1 (the problem) and section 2 (design principles). Stop and agree the problem is real before touching code.
2. Read section 4 (data pipeline) with `configs/features.yaml` open and `src/pgat_length/features/plans.py` next to it.
3. Read section 5 (encoder) with `src/pgat_length/models/tokenizer.py` next to it.
4. Skim sections 6, 7, 8 for the training story.
5. Read section 9 in full — evaluation is where the thesis is defended.
6. Use section 10 as a jump table.
7. Cite from section 12 when writing the thesis.

Appendix B — Common questions

Q: Why not use TimeSformer for the spatial view too? A: Because DINOv2 gives per-view static embeddings and PGAT-v1 established they work well. TimeSformer is used for motion because it captures short temporal windows, which is complementary.

Q: Why is $K_{\max} = 32$ and not 40? A: The pose-plan builder requires $K_{\text{temporal}} \cdot \text{min_segment_width} \leq P$. With $P = 64$ and $\text{min_segment_width} = 2$, $K_{\max} = 32$. DEV samples top out at $F \approx 250$ giving $K \approx 31$, so the cap is not restrictive.

Q: What happens if a DINOv2 crop is invalid? A: The corresponding view is zeroed and its validity flag is False. Downstream, the confidence-safe fallback (section 5.2) blends toward the global view.

Q: Why full mBART fine-tune and not LoRA? A: (a) mBART is 610M vs Qwen's 3B; VRAM-feasible on the A6000. (b) It's pretrained for the exact target-language task (German translation), so full fine-tune gives it the freedom to adapt fully to the visual-prefix input

distribution. (c) It matches GFSLT-VLP's decoder choice, making the architectural comparison honest.

Q: How do I add a new metric to the evaluation? A: Add it to

`src/pgat_length/evaluation/metrics.py::sample_metrics` and `corpus_metrics` , then rerun `scripts/07_evaluate_dev.py` . Bootstrap it in `bootstrap.py` if you want a paired CI. No training rerun needed.